

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт
з лабораторної роботи №3
з дисципліни: «Поглиблене вивчення JavaScript»
з теми: «Розробка менеджера задач із глобальним станом»

Виконав:
ст. гр. ПЗПІ-23-3
Білоус А. А.

Перевірів:
доц. каф. ПІ
Мар'їн С. О.

3 РОЗРОБКА МЕНЕДЖЕРА ЗАДАЧ ІЗ ГЛОБАЛЬНИМ СТАНОМ

3.1 Мета роботи

Формування практичних навичок організації глобального стану у Vue-застосунках за допомогою бібліотеки Pinia, опанування маршрутизації через Vue Router, розуміння принципів масштабування компонентної архітектури та побудови багатосторінкового SPA з централізованим керуванням даними.

3.2 Хід роботи

3.2.1 Структура проєкту

Застосунок побудовано з використанням Vite + Vue 3, Pinia та Vue Router. Файлова структура:

- «src/stores/tasks.js» – Pinia-стор із задачами;
- «src/router/index.js» – конфігурація маршрутів;
- «src/views/TaskListView.vue» – список задач із фільтрацією та пошуком;
- «src/views/TaskDetailView.vue» – деталі окремої задачі;
- «src/views/TaskFormView.vue» – форма створення та редагування задачі;
- «src/views/StatsView.vue» – сторінка статистики;
- «src/components/TaskItem.vue» – компонент рядка задачі;
- «src/App.vue» – кореневий компонент із навігацією та «<RouterView>».

3.2.2 Ініціалізація застосунку

У «main.js» до екземпляра Vue підключено Pinia та Router:

```
const app = createApp(App)
app.use(createPinia())
app.use(router)
app.mount('#app')
```

3.2.3 Pinia-стор

Стор визначено у Setup-стилі. Кожна задача містить поля «id», «title», «description», «category», «priority», «deadline» та «done». Стан відновлюється з «localStorage» при ініціалізації та зберігається через «watch»:

```

const saved = localStorage.getItem(STORAGE_KEY)
const tasks = ref(saved ? JSON.parse(saved) : [])

watch(tasks, val => {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(val))
}, { deep: true })

```

Getters формують похідні колекції без дублювання логіки в компонентах:

```

const activeTasks = computed(() => tasks.value.filter(t => !
t.done))
const overdueTasks = computed(() =>
  tasks.value.filter(t => !t.done && t.deadline && t.deadline <
todayStr())
)
const upcomingDeadlines = computed(() => {
  // задачі з дедлайном протягом 3 днів, відсортовані за датою
})
const stats = computed(() => ({
  total: tasks.value.length,
  active: activeTasks.value.length,
  done: doneTasks.value.length,
  overdue: overdueTasks.value.length,
}))

```

Actions: «addTask», «updateTask», «deleteTask», «toggleTask».

3.2.4 Vue Router

Маршрутизатор налаштовано з п'ятьма маршрутами. Статичний маршрут «/tasks/new» визначено перед динамічним «/tasks/:id», щоб уникнути хибного збігу:

```

const routes = [
  { path: '/', redirect: '/tasks' },
  { path: '/tasks', component: TaskListView },
  { path: '/tasks/new', component: TaskFormView },
  { path: '/tasks/:id', component: TaskDetailView },
  { path: '/tasks/:id/edit', component: TaskFormView },
  { path: '/stats', component: StatsView },
]

```

3.2.5 Сторінка списку задач

«TaskListView.vue» зберігає фільтри («categoryFilter», «priorityFilter», «sortBy», «search») як локальний стан компонента — вони не потребують глобального сховища. Відфільтрований та відсортований масив формується через «computed»:

```
const filteredTasks = computed(() => {
  let result = store.tasks
  if (search.value.trim()) result = result.filter(t =>
t.title.includes(...))
  if (categoryFilter.value) result = result.filter(t => t.category
=== categoryFilter.value)
  if (priorityFilter.value) result = result.filter(t => t.priority
=== priorityFilter.value)
  return [...result].sort(...)
})
```

3.2.6 Компонент TaskItem

«TaskItem.vue» отримує об'єкт задачі через props і звертається до стору напряду для виклику дій – без prop drilling. Прострочені задачі виділяються через реактивний «computed» та динамічний клас:

```
const isOverdue = computed(() =>
!props.task.done && !!props.task.deadline && props.task.deadline <
today
)

<li class="task-item" :class="{ overdue: isOverdue }">
```

3.2.7 Форма задачі

«TaskFormView.vue» використовується як для створення («/tasks/new»), так і для редагування («/tasks/:id/edit»). Режим визначається наявністю «route.params.id». При редагуванні форма заповнюється даними існуючої задачі через «onMounted». Валідація перевіряє обов'язкову назву та неприпустимість дедлайну в минулому при створенні.

3.2.8 Сторінка деталей

«TaskDetailView.vue» отримує «id» з «route.params», знаходить задачу в сторі. При видаленні викликає «store.deleteTask» та перенаправляє через «router.push('/tasks')». Якщо задачу не знайдено – відображається повідомлення про помилку.

3.2.9 Сторінка статистики

«StatsView.vue» використовує виключно getters сховища – жодної логіки в компоненті. Відображає загальну кількість задач за статусами та список задач із дедлайном протягом 3 днів.

3.3 Висновки

У ході лабораторної роботи розроблено повнофункціональний менеджер задач із п'ятьма маршрутами, категоріями, пріоритетами та дедлайнами. Pinia-стор у Setup-стилі інкапсулює всю логіку: реактивний стан, обчислювані колекції (активні, виконані, прострочені, найближчі дедлайни) та CRUD-дії. Стан зберігається в «localStorage» через «watch». Vue Router забезпечує навігацію між сторінками без перезавантаження, а спільна форма «TaskFormView» обслуговує і створення, і редагування задач залежно від параметрів маршруту.

3.4 Відповіді на контрольні запитання

- а) **Що таке prop drilling і яку проблему він спричиняє?** Prop drilling – передача даних через ланцюжок проміжних компонентів, які самі ці дані не використовують, лише передають далі. Це ускладнює підтримку: при зміні структури даних треба оновлювати кожен компонент у ланцюжку. Глобальний стан (Pinia) вирішує цю проблему – будь-який компонент звертається до стору напряду.
- б) **З яких трьох частин складається Pinia-сховище? Яке призначення кожної?** У Options-стилі: «state» (реактивні дані), «getters» (обчислювані значення на основі стану, аналог «computed»), «actions» (методи для зміни стану та асинхронні операції). У Setup-стилі ці ролі виконують «ref»/«reactive», «computed» та звичайні функції відповідно.
- в) **Як компонент отримує доступ до даних Pinia-сховища?** Викликом функції «useTasksStore()» всередині компонента. Повернений об'єкт є реактивним – зміни в сторі автоматично оновлюють шаблон компонента. Доступ отримується в будь-якому компоненті незалежно від його положення в дереві.

- г) **Що таке маршрут у Vue Router? Як визначаються динамічні параметри маршруту?** Маршрут – це об’єкт `{ path, component }`, що пов’язує URL-шаблон із компонентом. Динамічні параметри позначаються двокрапкою: `path: '/tasks/:id'`. Значення параметра доступне у компоненті через `useRoute().params.id`.
- д) **Яке призначення компонентів RouterView та RouterLink?** `<RouterView>` – точка вставки, куди Router рендерить компонент поточного маршруту. `<RouterLink to="...">` виконує клієнтську навігацію без перезавантаження і автоматично додає клас `router-link-active` до активного посилання.
- е) **Як отримати параметри поточного маршруту у компоненті?** За допомогою composable `useRoute()` з `vue-router`: `const route = useRoute()`. Після цього доступні `route.params`, `route.query`, `route.path` тощо. Всі поля реактивні.
- ж) **Як програмно перенаправити користувача на інший маршрут?** За допомогою composable `useRouter()`: `const router = useRouter()`, потім `router.push('/tasks')` або `router.replace('/tasks')`. `push` додає запис в історію, `replace` замінює поточний без можливості повернення назад.
- и) **Чим відрізняються getters від actions у Pinia-сховищі?** Getters – це `computed`-властивості: вони лише читають і обчислюють дані на основі стану, не змінюючи його. Actions – це функції, які можуть змінювати стан і виконувати асинхронні операції (наприклад, запити до API).
- к) **Як реалізувати збереження стану сховища між перезавантаженнями сторінки?** Вручну через `watch`: при кожній зміні масиву задач серіалізована копія записується у `localStorage`, а при ініціалізації стору – зчитується назад. Альтернатива – плагін `pinia-plugin-persistedstate`, який робить те саме автоматично з опцією `persist: true` у сторі.
- л) **Яку перевагу дає винесення логіки фільтрації у getters сховища порівняно з реалізацією у кожному компоненті окремо?** Єдине місце визначення логіки – при зміні правил фільтрації редагується лише стор. Getter кешується і перераховується лише при зміні залежностей, тоді

як дублювання «computed» у кількох компонентах збільшує кількість підписок та ускладнює підтримку.